🚀 模块深度剖析：server.js

模块位置：`backend/server.js`

🧱 第一步：业务黑盒（产品说明书视角）

1. **核心职责**：`server.js` 是整个后端的**总装配台 + 总调度入口**。它不负责具体 Modbus 协议细节，也不自己解析传感器数据，而是像嵌入式里的 `main()` 一样，把配置、设备池、轮询引擎、OTA 服务、HTTP API、WebSocket 通道全部拉起来，然后把它们接成一条可运行的数据链。

2. **输入**：
   - **静态配置**：`backend/config/devices.json` 里的设备列表、轮询参数、后端端口。
   - **环境变量**：如 `DEVICE_IP`、`PORT`、`FIRMWARE_PATH`、`LOCAL_IP`。
   - **前端请求**：浏览器发来的 HTTP 请求和 WebSocket 消息。
   - **子模块回调**：`PollingEngine` 采集到的数据、设备状态变化、OTA 文件读写结果。

3. **输出**：
   - **对下**：把设备配置交给 `DevicePool`，把定时采集和控制指令交给 `PollingEngine`，把固件下载交给 `OTAHandler`。
   - **对上**：通过 Express 暴露 `/api/*` 接口，通过 WebSocket 把设备数据推给前端。
   - **对运维**：在控制台打印启动信息、访问地址、错误日志。

4. **使用场景举例**：
   - 你上电启动后端，`server.js` 先读取配置，再把设备 TCP 连接池、OTA 文件服务器、轮询引擎、WebSocket 服务全都带起来。
   - 浏览器连上后，一边用 `/api/devices` 拉设备列表，一边通过 WebSocket 实时收 `sensor_data`。
   - 管理员点击"开风机"或"开始 OTA"，`server.js` 负责把这条前端消息翻译成对 `PollingEngine` 的函数调用。

> 💡 ***C 语言类比**：它就像你在裸机项目里写的 `main()`。`main()` 本身不是 ADC 驱动、不是 UART 驱动、也不是 PID 算法，但它负责 `SystemInit()`、`MX_USART_Init()`、`MX_TIM_Init()`、注册中断回调、最后让整个系统"活起来"。*

🦴 第二步：静态骨架（代码结构图）

模块骨架

```
server.js
├── 顶层初始化
│   ├── require('./Logger') + initLogger()
│   ├── require('dotenv').config()
│   └── require 各子模块
├── loadConfig()
```

```
│   └── 默认值 + devices.json + 环境变量覆盖
├── getLocalIP()
│   └── LOCAL_IP 优先，否则扫描网卡 IPv4
├── main()
│   ├── 创建设备池 DevicePool
│   ├── 创建 OTAHandler 并启动文件下载服务
│   ├── 创建 PollingEngine
│   ├── 创建 Express + HTTP Server + WebSocketManager
│   ├── 注册 PollingEngine / WebSocket 回调桥接
│   ├── 注册 REST API 和 OTA Router
│   ├── listen 共享端口
│   ├── 连接所有设备
│   ├── 启动轮询
│   └── 启动 WebSocket 心跳
├── process.on('uncaughtException')
├── process.on('unhandledRejection')
├── process.on('SIGINT')
└── main().catch(...)
```

核心状态 / 句柄

名称	所在位置	用途	C 语言类比
config	main() 局部变量	后端启动参数总表	system_config_t cfg
localIP	main() 局部变量	生成 OTA 下载地址、打印局域网访问地址	读取网卡寄存器后的 IP 缓存
devicePool	main() 局部变量	持有所有设备连接句柄	串口句柄数组 / Socket 句柄表
otaHandler	main() 局部变量	固件文件下载服务	IAP 下载服务任务
pollingEngine	main() 局部变量	周期采集、继电器控制、OTA 触发	定时器驱动的调度器
app	main() 局部变量	Express 路由容器	应用层协议分发器
server	main() 局部变量	HTTP 与 WebSocket 共用的底层监听器	TCP 监听 Socket
wsManager	main() 局部变量	浏览器实时双向通信	USART DMA + 中断消息通道
app.locals	Express 共享挂载点	给 Router 传运行时句柄	全局句柄注册表

核心函数

函数 / 钩子	作用	C 语言类比
`loadConfig()`	读取并合并配置源	从 Flash 读默认参数，再叠加 EEPROM / 跳帽配置
`getLocalIP()`	获取本机局域网 IP	读取网卡当前地址
`main()`	完成所有初始化与启动顺序	`main()` 主启动流程
`wsManager.onClientMessage = ...`	WebSocket 指令分发	命令解析中断回调
`app.get(...)` / `app.use(...)`	REST API 和静态资源入口	协议命令表 / URL 分发表
`process.on(...)`	全局异常、退出钩子	HardFault 日志钩子 / 关机中断

⚙️ 第三步：动态运转（生命周期与数据流）

1. 启动与循环机制

`server.js` 的启动点在文件底部：

```
main().catch(err => {
    console.error('[Error] Fatal error:', err);
    process.exit(1);
});
```

也就是说，Node.js 一加载完这个模块，就会立刻开始执行 `main()`。`main()` 不是并行乱起，而是严格按顺序 `await`：

1. 读配置。
2. 算本机 IP。
3. 创建设备池并注册设备。
4. 启动 OTA 文件服务。
5. 创建轮询引擎。
6. 创建 Express / HTTP Server / WebSocket。
7. 注册回调和 API。
8. 开始监听端口。
9. 主动连接所有设备。
10. 启动后台轮询和 WebSocket 心跳。

这段顺序非常像嵌入式里“先初始化外设，再开中断，再进主循环”的套路。真正进入“持续运转”后，系统主要靠下面 3 条后台生命线继续跑：

- `server.listen(...)`：负责 HTTP / WebSocket 接入。

- `pollingEngine.start()`：内部用 `setInterval` 周期采集设备。
- `wsManager.startHeartbeat()`：内部用 `setInterval` 做连接保活。

> 💡 ***C 语言类比***：*`main()` 本身更像"上电初始化阶段"；真正持续工作的，是后面被拉起来的定时器任务、网络监听任务和事件回调。相当于 `main()` 把一堆外设和 RTOS 任务都启动后，系统就交给事件和中断继续运行了。*

2. 数据桥接是怎么流动的

这个模块最核心的价值，不是自己做业务，而是**桥接**：

向下桥接：前端消息 → 设备控制

- 浏览器通过 WebSocket 发来 `relay_control`、`ota_start`、`get_devices`。
- `server.js` 在 `wsManager.onClientMessage` 里做 `switch(message.type)` 分发。
- 如果是控制类消息，它先在 `devicePool.getAllDevices()` 里按 IP 找设备，再调用：
  - `pollingEngine.controlRelay(device.key, ...)`
  - `pollingEngine.triggerOTA(device.key, ...)`
- 如果只是查询设备列表，就直接 `sendToClient()` 回包。

这里的本质是：**浏览器只懂 JSON 消息，GD32 只懂 Modbus 寄存器；`server.js` 就是协议翻译层的转接板。**

向上桥接：轮询结果 → 浏览器实时刷新

- `server.js` 给 `pollingEngine.onData` 赋值。
- 当 `PollingEngine` 采到 `sensorData` 后，会回调这个函数。
- `server.js` 再把 `deviceKey` 反查成设备 IP，交给 `wsManager.pushSensorData(device.ip, sensorData)` 广播。

这就像：

1. 底层驱动采完 ADC / Modbus 数据。
2. 中间层把"硬件句柄"翻译成"业务设备编号"。
3. 通信层再打包成上位机能看懂的 JSON。

Router 桥接：Express 路由 → 运行时句柄

`server.js` 把：

```
app.locals.pollingEngine = pollingEngine;
app.locals.wsManager = wsManager;
```

挂到 `app.locals` 上，这样 `api/ota.js` 这种独立 Router 文件也能拿到运行时对象，不需要重新 `require` 或新建实例。

> 💡 ***C 语言类比***：*`app.locals` 很像一个全局句柄表，例如：*
>
> ```
> struct {
>  PollingEngine *poll;
> ```

```
    WsManager *ws;
} g_app_handles;
```

> *路由文件就像别的 `.c` 模块，从这个全局表里取到运行时上下文。*

3. 异常与边界处理

已经做了的防线

场景	代码防线	C 语言类比
配置文件不存在 / JSON 解析失败	`loadConfig()` 有默认配置和 `try-catch`	EEPROM 读坏了就回退工厂参数
多网卡拿错 IP	`LOCAL_IP` 可手动覆盖	手动指定网口配置
WebSocket 指令异常	每个 `case` 自带 `try-catch` 并回发失败响应	串口命令执行失败时回 ACK/NAK
API 参数不全	Router 返回 400 / 500	命令字缺参直接报错
顶层 Promise 崩溃	`main().catch(...)` 统一兜底	启动阶段失败直接复位/退出
运行时未捕获异常	`process.on('uncaughtException')` / `unhandledRejection` 打日志	HardFault/Assert 记录器

需要你特别看清的真实边界

这几个点不是"语法错误"，但它们会影响你对真实运行效果的判断：

1. `pollingEngine.onStatusChange` **目前只是被赋值，没有被真正触发。**
   `server.js` 里确实写了：

   ```
   pollingEngine.onStatusChange = (deviceKey, status) => { ... }
   ```

   但 `PollingEngine.js` 内部当前只调用 `this.onData(...)`，没有调用 `this.onStatusChange(...)`。反而真正会产生状态事件的是 `DevicePool.emit('statusChange', ...)`。
   这意味着：**从代码现状看，设备状态推送这条链是"接了线，但上游还没送电"。**

2. **REST OTA 接口和** `PollingEngine` **的参数契约有错位。**
   `api/ota.js` 把 `deviceIp` 直接传给：

   - `pollingEngine.triggerOTA(deviceIp, ...)`
   - `pollingEngine.readOTAStatus(deviceIp)`

   但 `PollingEngine` 这些方法期望的是 `key = ip:port:unitId`，不是裸 IP。
   WebSocket 分支先做了 `find(...).key`，REST 分支却没有。
   这意味着：**同样叫"启动 OTA"，WebSocket 路径和 REST 路径实际上走的是两套不同的设备定位逻辑。**

3. **日志里打印的 OTA 端口和实际启动端口不一致。**
   `server.js` 创建 `OTAHandler` 时端口写的是 `18080`，但启动完成日志打印的是：

   ```
   http://${localIP}:8080/download/SciGeneAI.rbl
   ```

   所以真实的固件服务地址要以 `OTAHandler` 实例端口为准，而不是日志里这行字面值。

4. `SIGINT` **退出是“直接断电式退出”，不是“逐模块优雅停机”。**
   当前只做了：

   ```
   process.exit(0);
   ```

   没有显式调用 `pollingEngine.stop()`、`wsManager.stop()`、`otaHandler.stopServer()`、`devicePool.disconnect()`。
   这更像 MCU 被直接断电，而不是先走一遍 `deinit()`。

> 💡 ***C 语言类比**：这几类问题都属于“接口契约没完全对齐”。就像一个模块以为自己拿到的是 `UART_HandleTypeDef*`，另一个模块却只传了串口号；编译不一定报错，但运行行为会跑偏。*

🎯 第四步：执行主线

① Happy Path（顺风局：后端从启动到进入稳定运行）

下面这段就是 `server.js` 的“主上电路径”，我把关键动作都写成中文步骤注释了：

```
async function main() {
    // 步骤1：读取配置，把默认值、devices.json、环境变量揉成最终启动参数
    const config = loadConfig();
    const localIP = getLocalIP();

    // 步骤2：创建设备池，把每台 GD32 设备注册成一个连接句柄
    const devicePool = new DevicePool();
    for (const dev of config.devices) {
        if (dev.enabled) {
            devicePool.addDevice(dev);
        }
    }

    // 步骤3：先启动 OTA 文件服务器，让设备将来有固件可下载
    const otaHandler = new OTAHandler({
        firmwarePath: config.backend.firmwarePath,
        backendIp: localIP,
        port: 18080
    });
    await otaHandler.startServer();
```

```
    // 步骤4：把“轮询引擎”装上设备池
    const pollingEngine = new PollingEngine(devicePool, config.polling);

    // 步骤5：搭 HTTP 外壳，并在同一个 server 上挂 WebSocket
    const app = express();
    const server = http.createServer(app);
    const wsManager = new WebSocketManager({ server });

    // 步骤6：注册桥接回调，让底层采集结果能推到浏览器
    pollingEngine.onData = (deviceKey, sensorData) => {
        const device = devicePool.getAllDevices().find(d => d.key === deviceKey);
        if (device) {
            wsManager.pushSensorData(device.ip, sensorData);
        }
    };

    // 步骤7：注册 WebSocket 消息分发和 REST API
    wsManager.onClientMessage = async (clientId, message) => {
        // 浏览器来的 JSON 命令，在这里被翻译成后端函数调用
    };
    app.locals.pollingEngine = pollingEngine;
    app.use('/api/ota', createOtaRouter(otaHandler));

    // 步骤8：正式监听端口，开始接收浏览器访问
    server.listen(config.backend.port || 3000);

    // 步骤9：主动连所有设备，再启动后台轮询和心跳
    for (const device of devicePool.getAllDevices()) {
        if (device.enabled) {
            await devicePool.connect(device.key);
        }
    }
    pollingEngine.start();
    wsManager.startHeartbeat();
}
```

这条 Happy Path 可以概括成一句话：

先把“能说话的人”都创建出来，再把“说话的线路”接起来，最后统一送电开跑。

② Edge Cases（逆风局：防御性处理和真实风险）

- **配置缺失时不停机，而是退回默认参数。**
  这样项目第一次拉起时，即使 `devices.json` 还没准备好，系统也不会一上来就崩。
  原因：这像工厂默认参数区，先保证板子能起来。

- **WebSocket 控制消息每个分支都 `try-catch`，失败时回响应。**
  这样前端不会一直傻等，也能知道是“设备没找到”还是“写寄存器失败”。

原因：这相当于命令总线上的应答帧。

- **全局异常只做记录，不做自动恢复。**
  `uncaughtException` / `unhandledRejection` 当前主要用于打印日志。
  原因：它更像事故黑匣子，不是热备切换模块。

- **退出路径没有逐模块收尾。**
  `SIGINT` 直接 `process.exit(0)` ，后台定时器和 Socket 没有显式 stop。
  原因：当前实现更偏“开发期快速退出”，不像工业控制里那种严格的停机序列。

- **设备状态推送链存在“线接好了，但上游没发事件”的情况。**
  `pollingEngine.onStatusChange` 被赋值，但 `PollingEngine` 里没实际调用。
  原因：设计意图和当前实现还没有完全闭环。

- **REST OTA 路径的设备标识与轮询引擎契约不一致。**
  WebSocket 分支传 `device.key` ，REST 分支传 `deviceIp` 。
  原因：两个入口都想控制同一个底层模块，但只对齐了一半协议。

- **日志里的 OTA 地址和真实监听端口不一致。**
  端口常量写死在日志里，容易误导排障。
  原因：这像串口调试打印没跟着宏定义改，系统实际跑得没错，但说明书写错了。

③ 关键语法点拨（跨界降维打击）

> 💡 *`async/await`*
> *在这里它就像 C/RTOS 里的 `xSemaphoreTake(sem, portMAX_DELAY)` 。 `await otaHandler.startServer()` 的意思不是“CPU 卡死”，而是“这一步没完成前，不继续初始化下一步”。*

> 💡 ***扩展运算符*** *`{ ...config, ...fileConfig }`*
> *这就像 C 里先 `memcpy` 一份默认结构体，再把用户配置字段覆写进去。前面的默认值保底，后面的文件值优先。*

> 💡 ***可选链*** *`config.devices?.length` / `config.devices[0]?.ip`*
> *相当于：*
>
> ```
> if (config.devices && config.devices[0]) { ... }
> ```
>
> *它的作用就是防空指针，防止某一层不存在时直接崩。*

> 💡 ***箭头函数*** *`(...) => { ... }`*
> *这里最重要的不是“短”，而是它天然抓住了外层的 `devicePool` 、 `wsManager` 、 `pollingEngine` 。*
> *这有点像把几个外部句柄打包进一个回调函数指针上下文里，回调触发时不用再到处全局查找。*

> 💡 *`app.locals`*
> *它相当于 Express 世界里的“全局运行时句柄表”。别的 Router 文件不需要 new 一个新的引擎实例，直接从这里取现成的就行。*

一句话收尾

`server.js` 不像一个“干重活的业务类”，它更像一个**系统集成总控文件。**你理解它的关键，不是盯着某一行API，而是看清它做了三件事：

1. **先装配**：把设备池、OTA、轮询、HTTP、WebSocket 都创建起来。
2. **再接线**：把模块间的回调、路由、共享句柄串通。
3. **最后通电**：监听端口、连设备、开轮询、开心跳。

如果用嵌入式思维来记，这个文件最像：

`main.c` **+ 模块初始化表 + 回调绑定区 + 系统启动顺序表。**